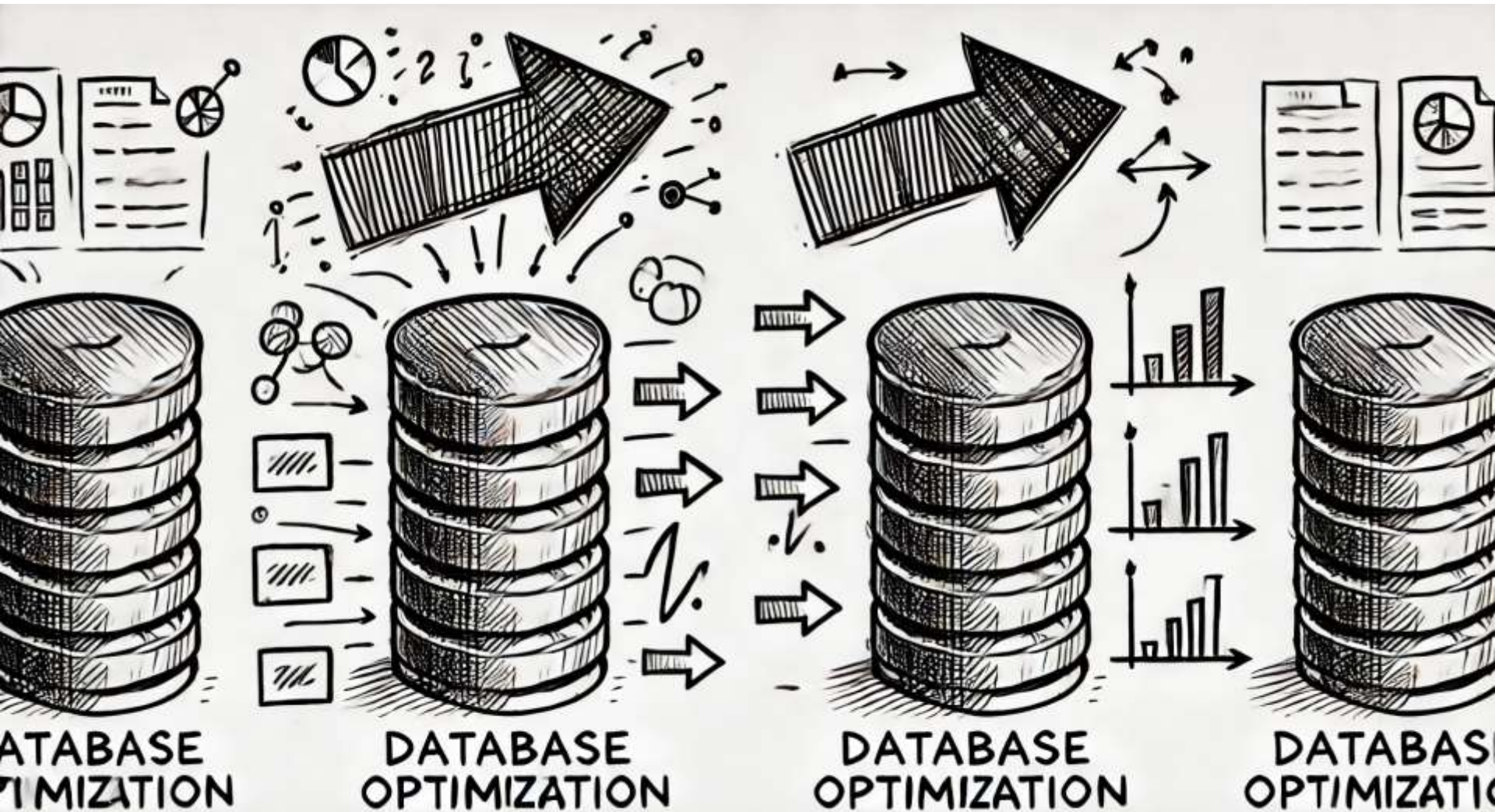
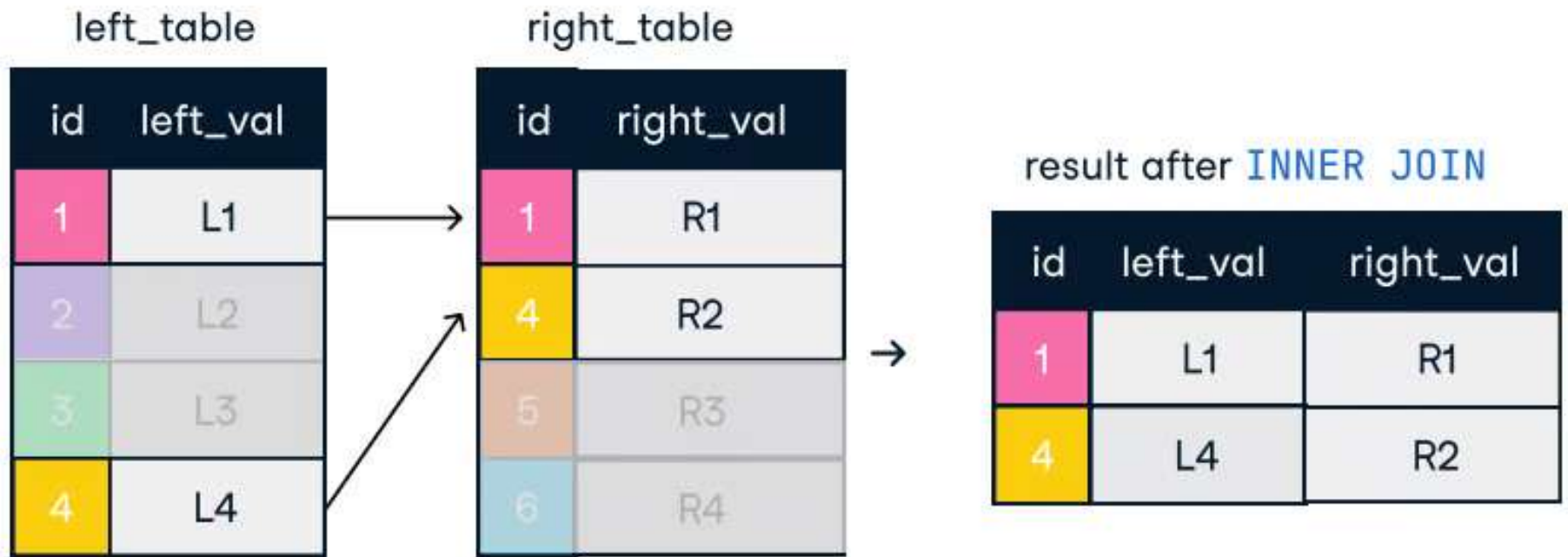


Query Optimization



Join



Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
 - Note: order of tuples is irrelevant
 - we don't care if they generate different results on databases that violate integrity constraints
- In SQL, inputs and outputs are multisets of tuples
 - Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of first form by second, or vice

Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$s_{q_1 \cup q_2}(E) = s_{q_1}(s_{q_2}(E))$$

2. Selection operations are commutative.

$$s_{q_1}(s_{q_2}(E)) = s_{q_2}(s_{q_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.

- a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

- b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6. (a) Natural join operations are associative:

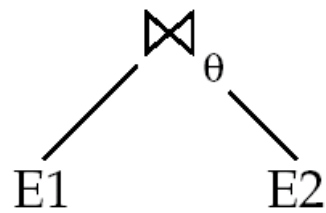
$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- (b) Theta joins are associative in the following manner:

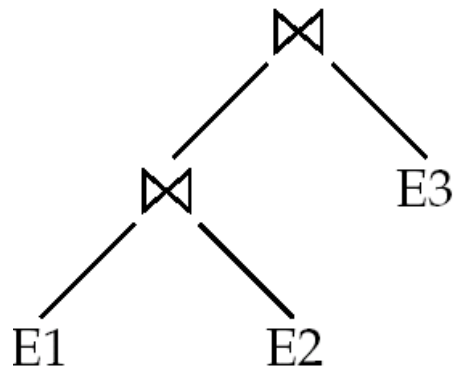
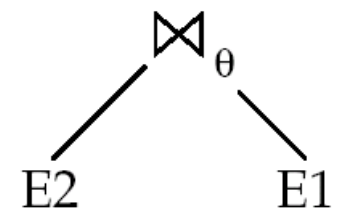
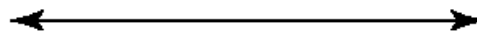
$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

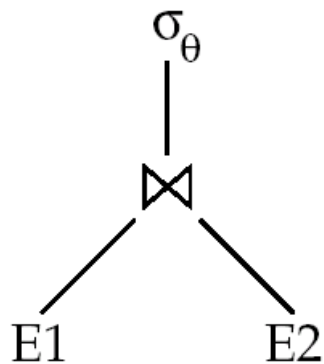
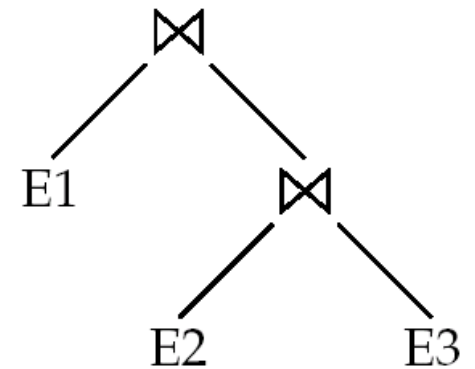
Pictorial Depiction of Equivalence Rules



Rule 5



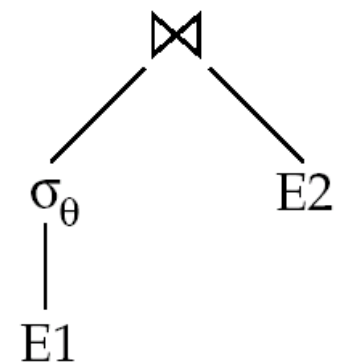
Rule 6a



Rule 7a



If θ only has
attributes from E1



Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

(b) Consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

□ (set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta} (E_1) - \sigma_{\theta}(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

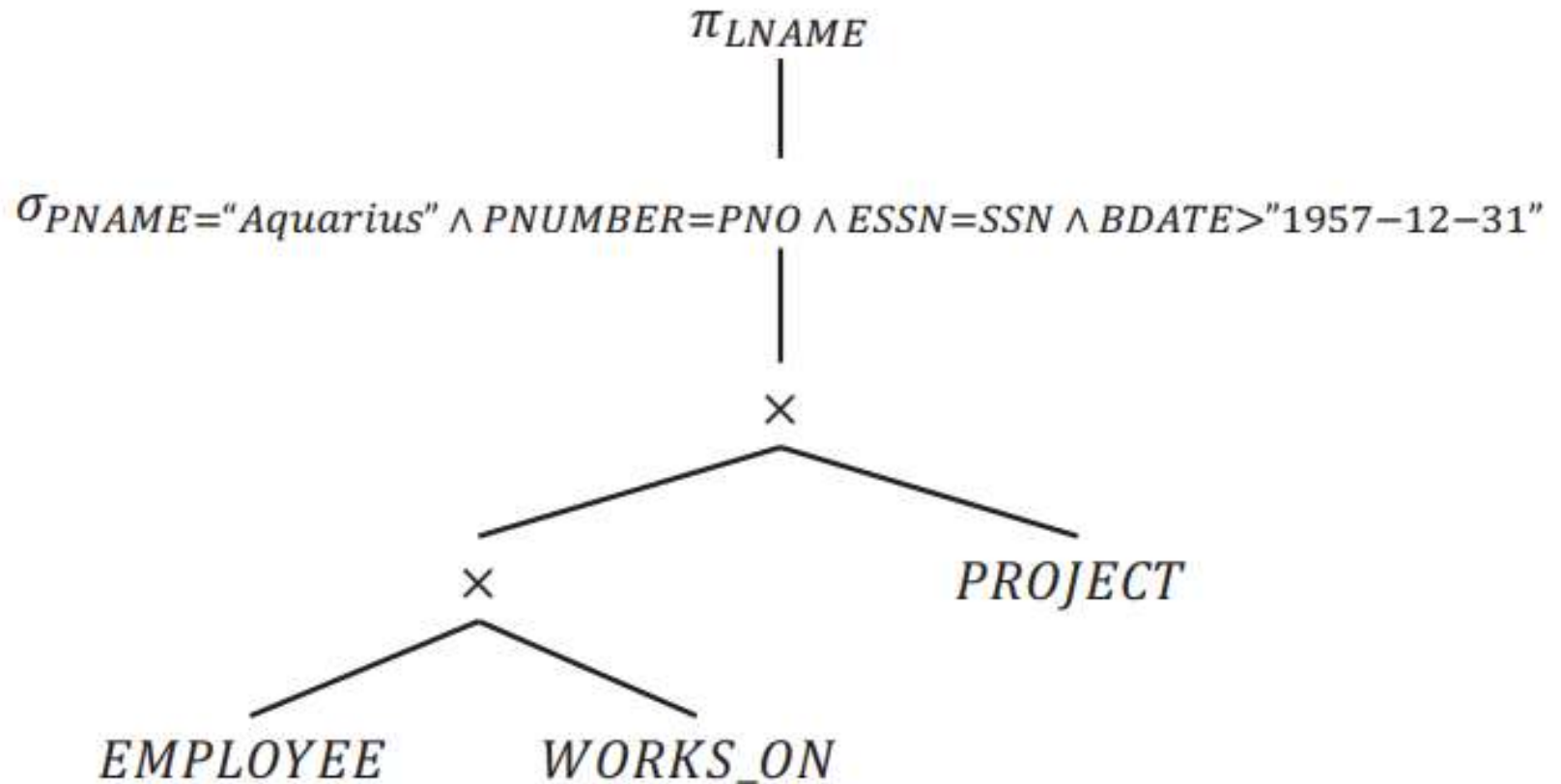
Also:
$$\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

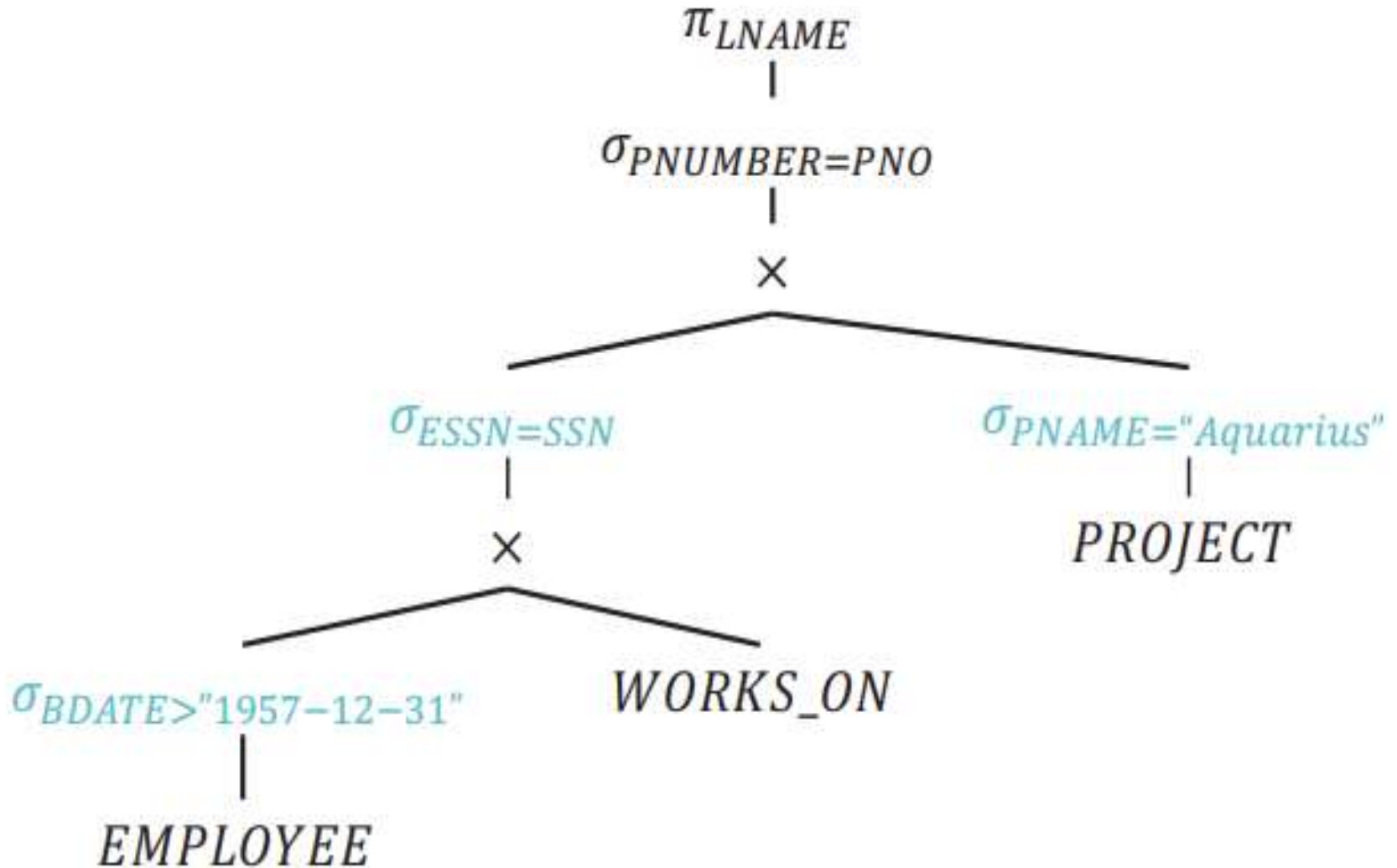
12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

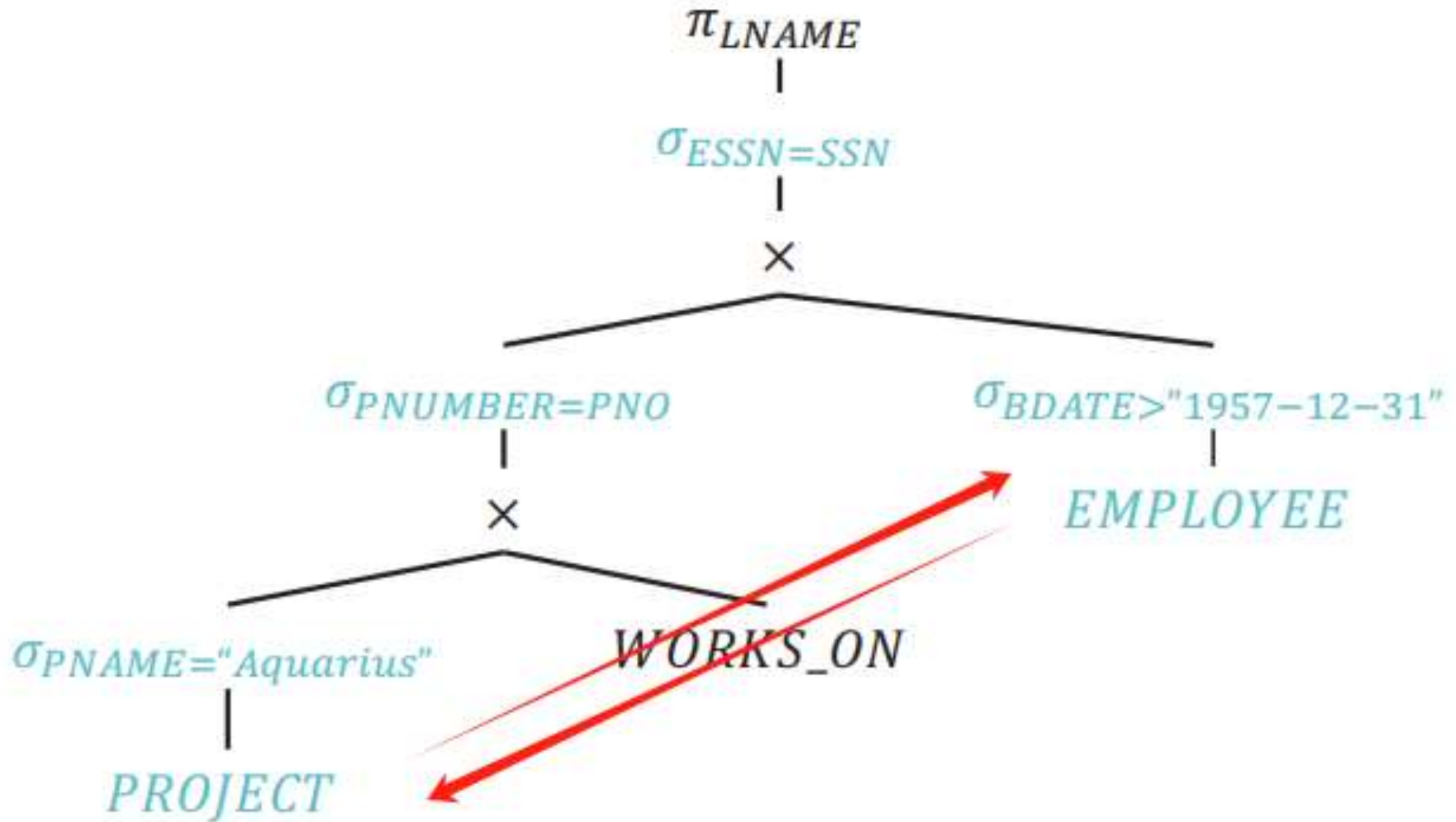
Example



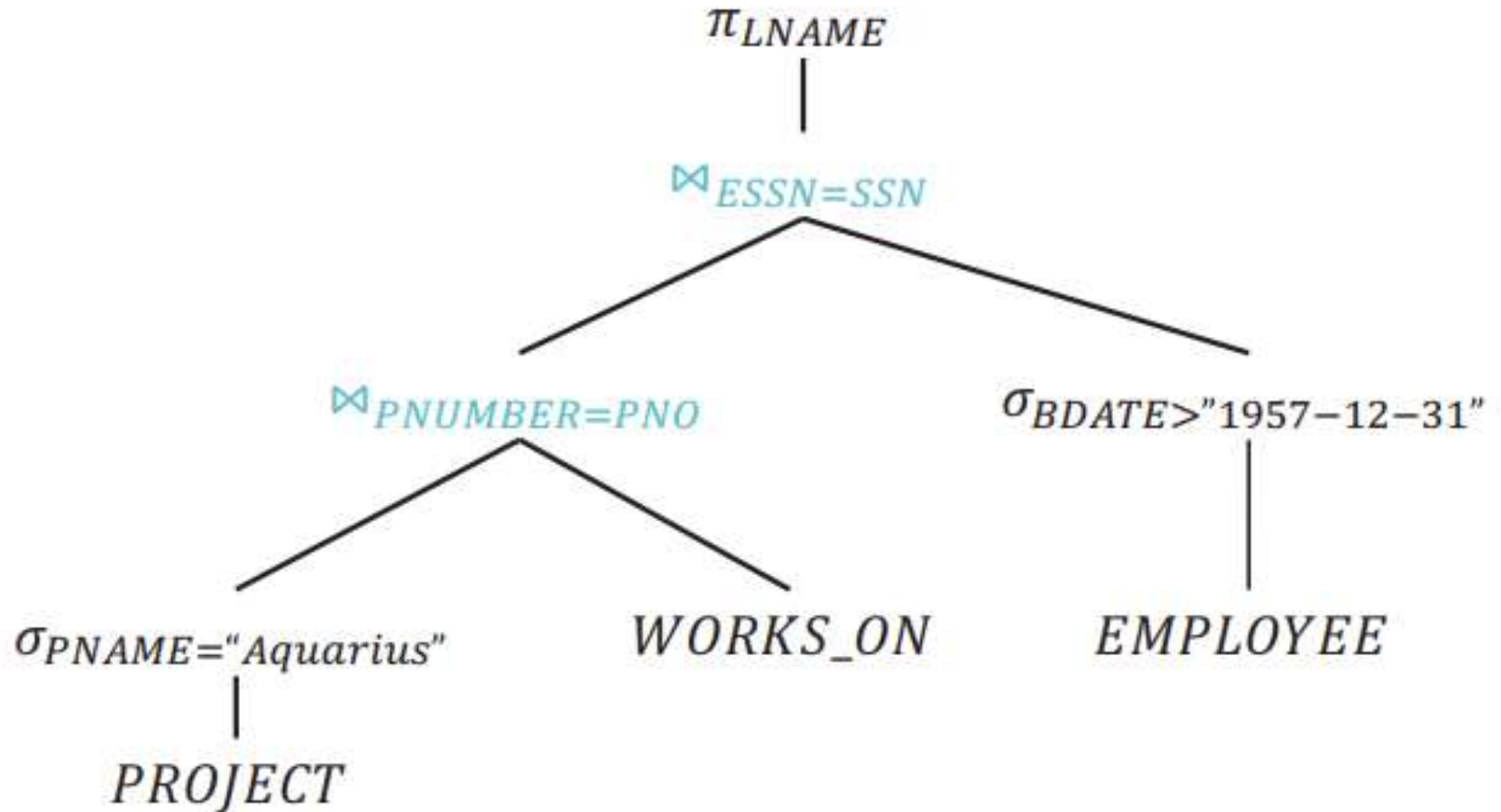
Move σ down



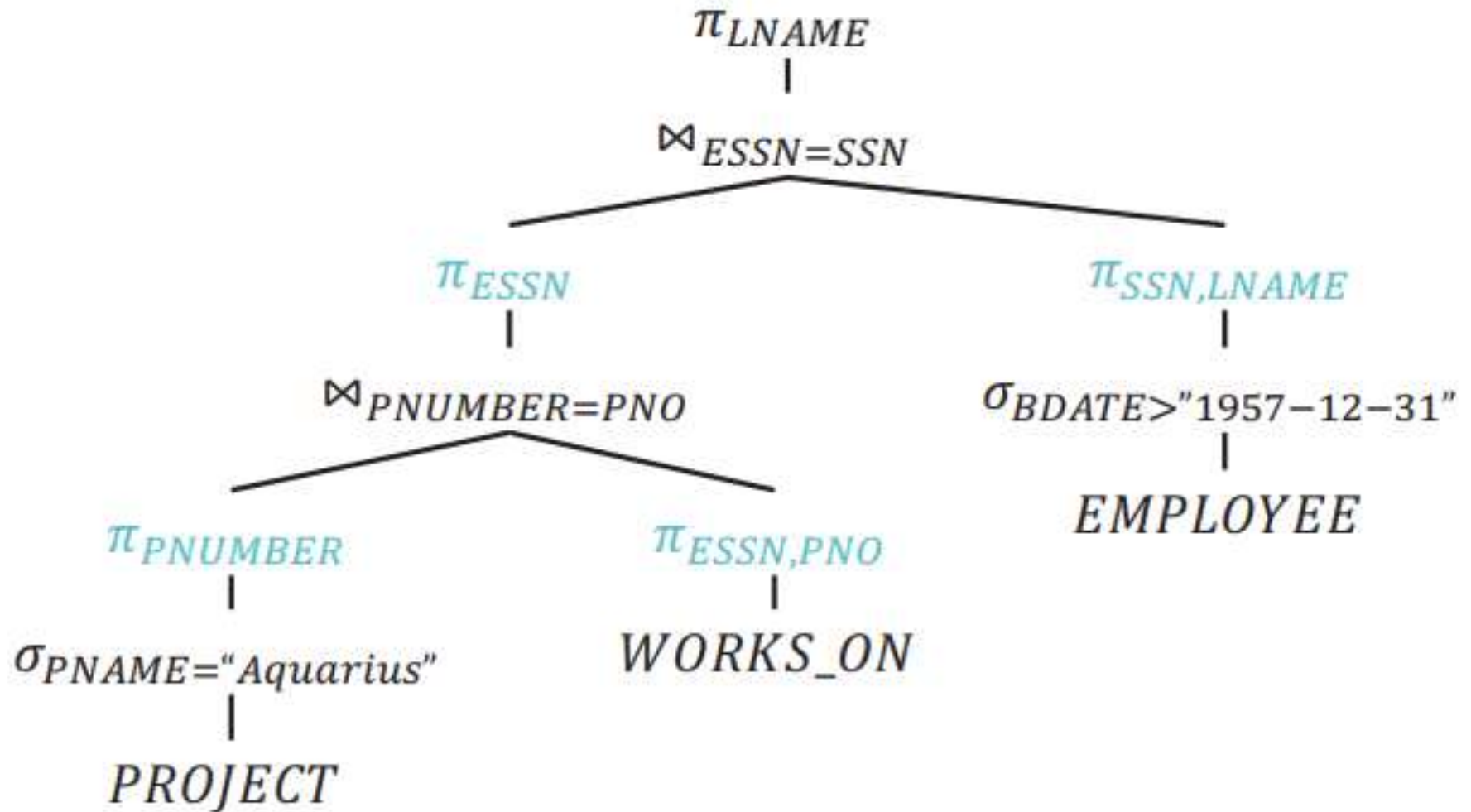
Reorder Joins



Combine Products to Create joins



Move π down



Transformation Example: Pushing Selections

- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach

- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$

- Transformation .

- $\Pi_{name, title}((\sigma_{dept_name = \text{"Music"}}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$

- Performing the selection as early as possible reduces the size of the relation to be joined.

Example with Multiple Transformations

- Query: Find the names of all instructors in the Music department who have taught a course in 2009, along with the titles of the courses that they taught

- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2009} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title} (course))))$

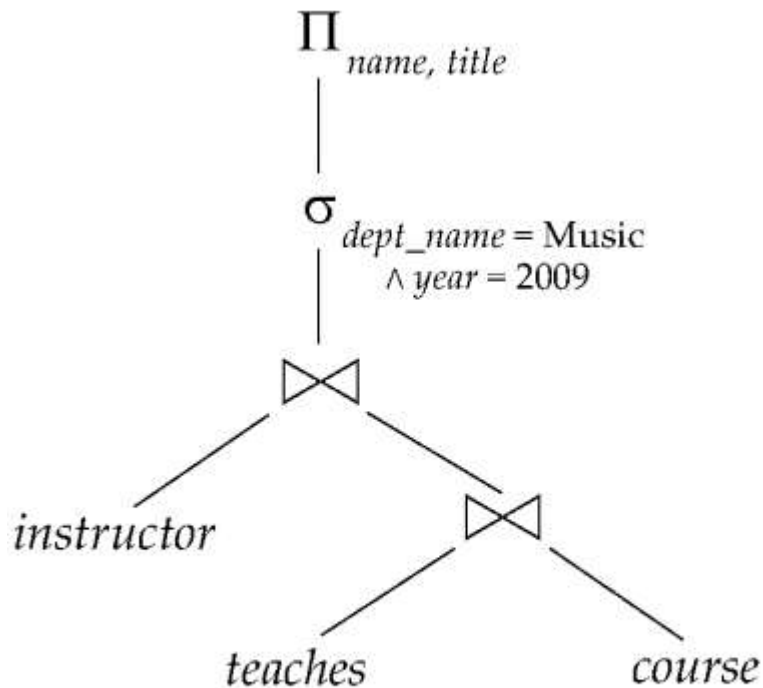
- Transformation using join associatively :

- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2009} ((instructor \bowtie teaches) \bowtie \Pi_{course_id, title} (course)))$

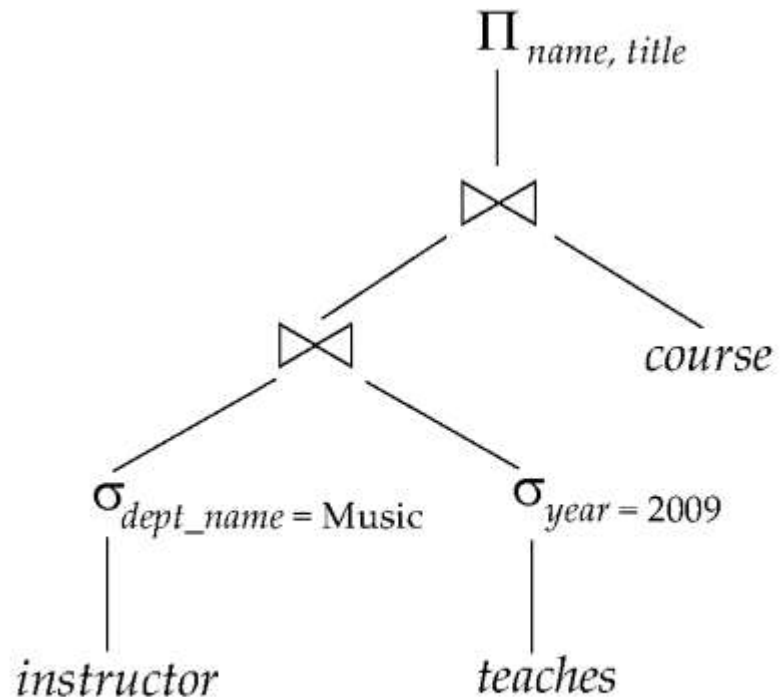
- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

- $\sigma_{dept_name = \text{"Music"}} (instructor) \bowtie \sigma_{year = 2009} (teaches)$

Multiple Transformations (Cont.)



(a) Initial expression tree



(b) Tree after multiple transformations

Transformation Example: Pushing Projections

- Consider: $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$

- When we compute

$$(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches)$$

we obtain a relation whose schema is:

$(ID, name, dept_name, salary, course_id, sec_id, semester, year)$

- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course)))$$

- Performing the projection as early as possible reduces the size of the relation to be joined.

Join Ordering Example

- For all relations r_1 , r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite **large** and $r_1 \bowtie r_2$ is **small**, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name, title}(\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{“Music”}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
 - it is better to compute

$$\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches$$

first.

Optimization Rules

- Rule 1: Decomposing Conjunctive Selection
- Rule 2: Reordering Selection and Projection
- Rule 3: Join Optimization
- Rule 4: Pushing Down Selections
 - ▶ moves filtering as close as possible to the data source
- Rule 5: Join Algorithm Selection
 - ▶ small tables or indexes exist

Dynamic Programming in Optimization

- To find best join tree for a set of n relations:
 - To find best plan for a set S of n relations, consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S .
 - Recursively compute costs for joining subsets of S to find the cost of each plan. Choose the cheapest of the $2^n - 2$ alternatives.
 - Base case for recursion: single relation access plan
 - ▶ Apply all selections on R_i using best choice of indices on R_i
 - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it
 - ▶ Dynamic programming

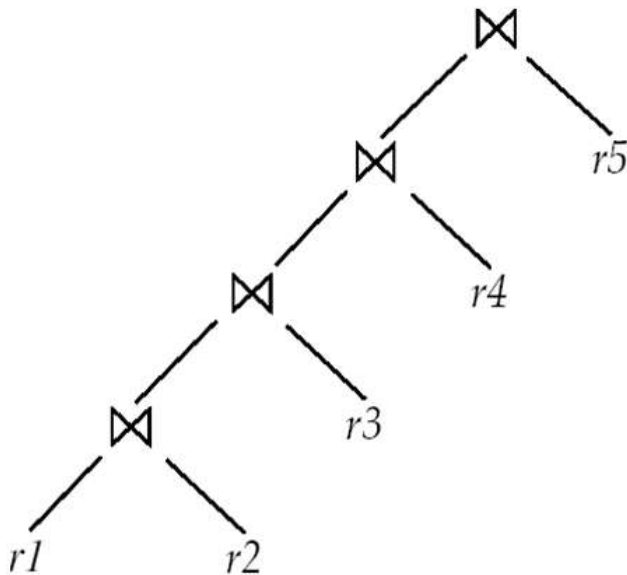
Join Order Optimization Algorithm

```
procedure findbestplan(S)
  if (bestplan[S].cost  $\neq \infty$ )
    return bestplan[S]
  // else bestplan[S] has not been computed earlier, compute it now
  if (S contains only 1 relation)
    set bestplan[S].plan and bestplan[S].cost based on the best way
    of accessing S /* Using selections on S and indices on S */
  else for each non-empty subset S1 of S such that S1  $\neq$  S
    P1= findbestplan(S1)
    P2= findbestplan(S - S1)
    A = best algorithm for joining results of P1 and P2
    cost = P1.cost + P2.cost + cost of A
    if cost < bestplan[S].cost
      bestplan[S].cost = cost
      bestplan[S].plan = “execute P1.plan; execute P2.plan;
                          join results of P1 and P2 using A”
  return bestplan[S]
```

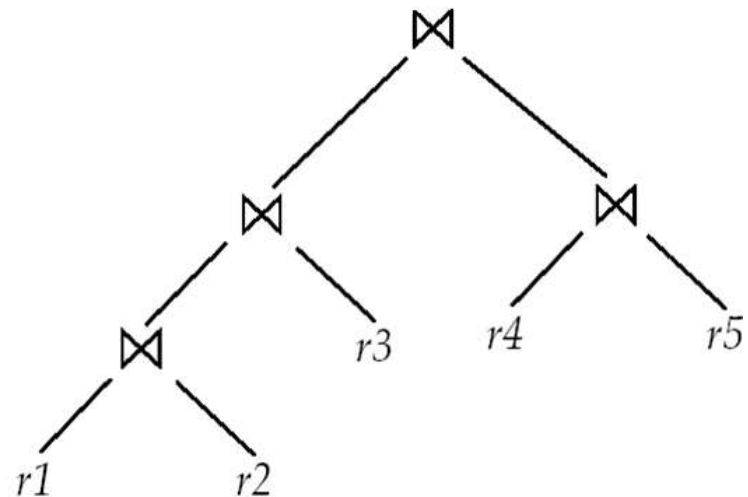
* Some modifications to allow indexed nested loops joins on relations that have selections (see book)

Left Deep Join Trees

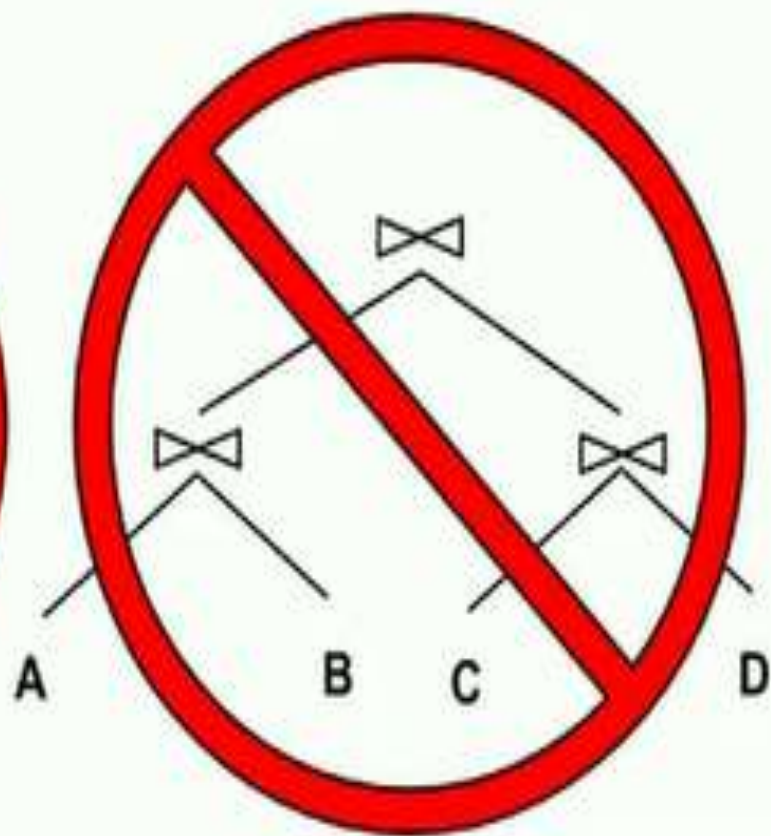
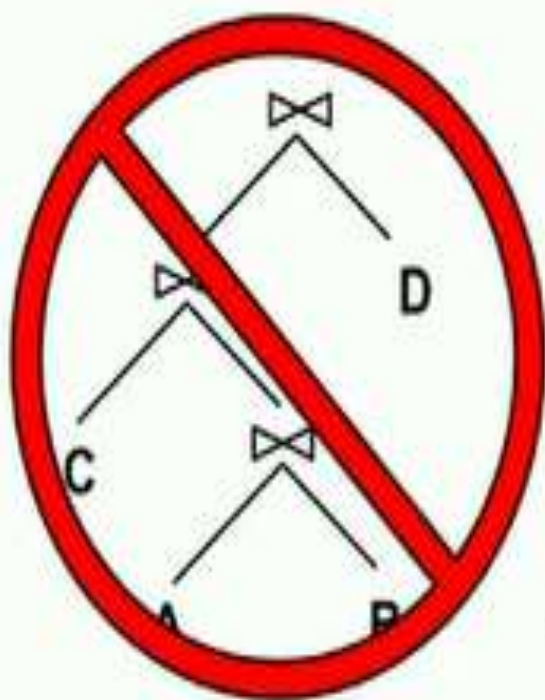
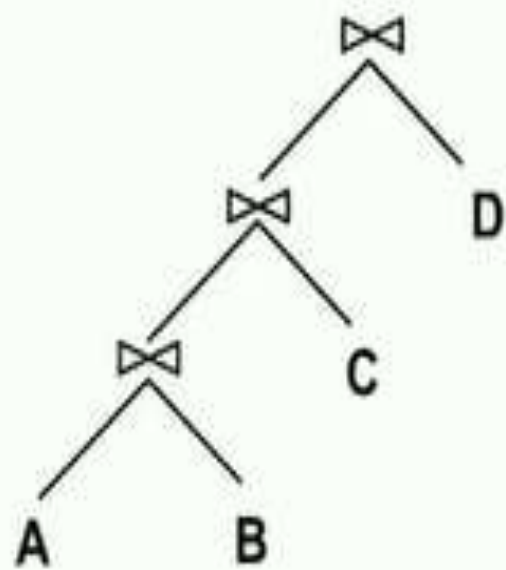
- In **left-deep join trees**, the right-hand-side input for each join is a relation, not the result of an intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree



Cost of Optimization

- With dynamic programming time complexity of optimization with bushy trees is $O(3^n)$.
 - With $n = 10$, this number is 59000 instead of 176 billion!
- Space complexity is $O(2^n)$
- To find best left-deep join tree for a set of n relations:
 - Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.
 - Modify optimization algorithm:
 - ▶ Replace “**for each** non-empty subset $S1$ of S such that $S1 \neq S$ ”
 - ▶ By: **for each** relation r in S
let $S1 = S - r$.
- If only left-deep trees are considered, time complexity of finding best join order is $O(n 2^n)$
 - Space complexity remains at $O(2^n)$
- Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n , generally < 10)

Interesting Sort Orders

- Consider the expression $(r_1 \bowtie r_2) \bowtie r_3$ (with A as common attribute)
- An **interesting sort order** is a particular sort order of tuples that could be useful for a later operation
 - Using merge-join to compute $r_1 \bowtie r_2$ may be costlier than hash join but generates result sorted on A
 - Which in turn may make merge-join with r_3 cheaper, which may reduce cost of join with r_3 and minimizing overall cost
 - Sort order may also be useful for order by and for grouping
- Not sufficient to find the best join order for each subset of the set of n given relations
 - must find the best join order for each subset, **for each interesting sort order**
 - Simple extension of earlier dynamic programming algorithms
 - Usually, number of interesting orders is quite small and doesn't affect time/space complexity significantly

Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples)
 - Perform projection early (reduces the number of attributes)
 - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
 - Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Optimizing Nested Subqueries**

- Nested query example:
select *name*
from *instructor*
where exists (**select** *
 from *teaches*
 where *instructor.ID = teaches.ID and teaches.year = 2007*)
- SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values
 - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause
 - Such evaluation is called **correlated evaluation**
 - Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery

Optimizing Nested Subqueries (Cont.)

- Correlated evaluation may be quite inefficient since
 - a large number of calls may be made to the nested query
 - there may be unnecessary random I/O as a result
- SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques
- E.g.: earlier nested query can be rewritten as
select *name*
from *instructor, teaches*
where *instructor.ID = teaches.ID and teaches.year = 2007*
 - Note: the two queries generate different numbers of duplicates (why?)
 - ▶ teaches can have duplicate IDs
 - ▶ Can be modified to handle duplicates correctly as we will see
- In general, it is not possible/straightforward to move the entire nested subquery from clause into the outer level query from clause
 - A temporary relation is created instead, and used in body of outer level query

Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

□ Rewrite: **select ...**

from L_1

where P_1 and exists (select *
from L_2
where P_2)

□ To: **create table t_1 as**
select distinct V
from L_2
where P_2^1

select ...
from L_1, t_1
where P_1 and P_2^2

- P_2^1 contains predicates in P_2 that do not involve any correlation variables
- P_2^2 reintroduces predicates involving correlation variables, with relations renamed appropriately
- V contains all attributes used in predicates with correlation variables

Optimizing Nested Subqueries (Cont.)

- In our example, the original nested query would be transformed to
create table t_1 as
select distinct ID
from $teaches$
where $year = 2007$

select $name$
from $instructor, t_1$
where $t_1.ID = instructor.ID$

- The process of replacing a nested query by a query with a join (possibly with a temporary relation) is called **decorrelation**.
- Decorrelation is more complicated when
 - the nested subquery uses aggregation, or
 - when the result of the nested subquery is used to test for equality, or
 - when the condition linking the nested subquery to the other query is **not exists**,
 - and so on.

Materialized Views**

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view
create view *department_total_salary*(*dept_name*,
total_salary) **as**
select *dept_name*, **sum**(*salary*)
from *instructor*
group by *dept_name*
- Materializing the above view would be very useful if the total salary by department is **required frequently**
 - Saves the effort of finding multiple tuples and adding up their amounts



Changes to the table



Table

Materialized
view



View calculation



Materialized Views

Improved query performance

Reduced computational load

Enhanced data availability

Materialized View Maintenance

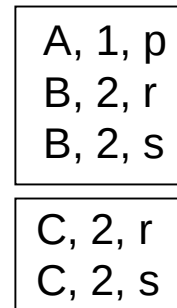
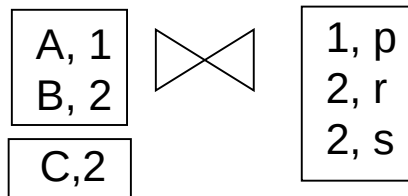
- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
 - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Above methods are directly supported by many database systems
 - ▶ Avoids manual effort/correctness issues

Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
 - Set of tuples inserted to and deleted from r are denoted $\mathbf{i}_r$ and $\mathbf{d}_r$
- To simplify our description, we only consider inserts and deletes
 - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions

Join Operation

- Consider the materialized view $v = r \bowtie s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insert to r :
 - We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
 - And rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
 - But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$



Selection and Projection Operations

- Selection: Consider a view $v = \sigma_{\theta}(r)$.
 - $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$
 - $v^{new} = v^{old} - \sigma_{\theta}(d_r)$
- Projection is a more difficult operation
 - $R = (A,B)$, and $r(R) = \{ (a,2), (a,3) \}$
 - $\Pi_A(r)$ has a single tuple (a) .
 - If we delete the tuple $(a,2)$ from r , we should not delete the tuple (a) from $\Pi_A(r)$, but if we then delete $(a,3)$ as well, we should delete the tuple
- For each tuple in a projection $\Pi_A(r)$, we will keep a count of how many times it was derived
 - On insert of a tuple to r , if the resultant tuple is already in $\Pi_A(r)$ we increment its count, else we add a new tuple with count = 1
 - On delete of a tuple from r , we decrement the count of the corresponding tuple in $\Pi_A(r)$
 - ▶ if the count becomes 0, we delete the tuple from $\Pi_A(r)$

Aggregate Operations (Cont.)

- **min, max:** $v = \mathcal{A}_{g_{min}(B)}(r)$.
- Handling insertions on r is straightforward.
- Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of r that are in the same group to find the new minimum

Aggregation Operations

- $\text{count} : v = {}_A \mathbf{g}_{\text{count}(B)}^{(r)}.$
 - When a set of tuples i_r is inserted
 - ▶ For each tuple r in i_r , if the corresponding group is already present in v , we increment its count, else we add a new tuple with count = 1
 - When a set of tuples d_r is deleted
 - ▶ for each tuple t in i_r , we look for the group $t.A$ in v , and subtract 1 from the count for the group.
 - If the count becomes 0, we delete from v the tuple for the group $t.A$
- $\text{sum} : v = {}_A \mathbf{g}_{\text{sum}(B)}^{(r)}$
 - We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count
 - Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v
 - ▶ Cannot simply test for $\text{sum} = 0$ (why?)
- To handle the case of **avg**, we maintain the **sum** and **count** aggregate values separately, and divide at the end

General Transformation Rules for Relational Algebra Operations:

1. Cascade of s: A conjunctive selection condition can be broken up into a cascade (sequence) of individual s operations:
$$s_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } cn}(R) = s_{c1}(s_{c2}(\dots(s_{cn}(R))\dots))$$
2. Commutativity of s: The s operation is commutative:
$$s_{c1}(s_{c2}(R)) = s_{c2}(s_{c1}(R))$$
3. Cascade of p: In a cascade (sequence) of p operations, all but the last one can be ignored:
$$p_{List1}(p_{List2}(\dots(p_{Listn}(R))\dots)) = p_{List1}(R)$$
4. Commuting s with p: If the selection condition c involves only the attributes A1, ..., An in the projection list, the two operations can be commuted:
$$p_{A1, A2, \dots, An}(s_c(R)) = s_c(p_{A1, A2, \dots, An}(R))$$

5. Commutativity of $\bowtie$ (and $\times$): The operation is commutative as is the $\times$ operation: $R \bowtie_c S = S \bowtie_c R$; $R \times S = S \times R$
6. Commuting s with $\bowtie$ (or $\times$): If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R —the two operations can be commuted as follows :

$$s_c (R \bowtie S) = (s_c (R)) \bowtie S$$

Alternatively, if the selection condition c can be written as $(c1 \text{ and } c2)$, where condition $c1$ involves only the attributes of R and condition $c2$ involves only the attributes of S , the operations commute as follows:

$$s_c (R \bowtie S) = (s_{c1} (R)) \bowtie (s_{c2} (S))$$

7. Commuting p with $\bowtie$ (or x): Suppose that the projection list is $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, where $A_1, \dots, A_n$ are attributes of R and $B_1, \dots, B_m$ are attributes of S . If the join condition c involves only attributes in L , the two operations can be commuted as follows:

$$p_L (R \bowtie_C S) = (p_{A_1, \dots, A_n} (R)) \bowtie_C (p_{B_1, \dots, B_m} (S))$$

If the join condition c contains additional attributes not in L , these must be added to the projection list, and a final p operation is needed.

8. Commutativity of set operations: The set operations $\cup$ and $\cap$ are commutative but $-$ is not.
9. Associativity of $\bowtie$, $\times$, $\cup$, and $\cap$: These four operations are individually associative; that is, if θ stands for any one of these four operations (throughout the expression), we have

$$(R \theta S) \theta T = R \theta (S \theta T)$$
10. Commuting σ with set operations: The σ operation commutes with $\cup$, $\cap$, and $-$. If θ stands for any one of these three operations, we have

$$\sigma_c (R \theta S) = (\sigma_c (R)) \theta (\sigma_c (S))$$

11. The π operation commutes with $\cup$.

$$\pi_L (R \cup S) = (\pi_L (R)) \cup (\pi_L (S))$$

12. Converting a (σ, x) sequence into $\bowtie$: If the condition c of a σ that follows a x Corresponds to a join condition, convert the (σ, x) sequence into a $\bowtie_c$ as follows:

$$(\sigma_c (R \times S)) = (R \bowtie_c S)$$

13. Other transformations

5. Using rules 3, 4, 7, and 11 concerning the cascading of project and the commuting of project with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new project operations as needed.
6. Identify subtrees that represent groups of operations that can be executed by a single algorithm.

Top-K Queries

□ Top-K queries

```
select *  
from r, s  
where r.B = s.B  
order by r.A ascending  
limit 10
```

- Alternative 1: Indexed nested loops join with r as outer
- Alternative 2: estimate highest r.A value in result and add selection (**and** r.A $\leq$ H) to where clause
 - ▶ If < 10 results, retry with larger H

Optimization of Updates

□ Halloween problem

**update R set $A = 5 * A$
where $A > 10$**

- If index on A is used to find tuples satisfying $A > 10$, and tuples updated immediately, same tuple may be found (and updated) multiple times
- Solution 1: *Always defer updates*
 - ▶ collect the updates (old and new values of tuples) and update relation and indices in second pass
 - ▶ Drawback: extra overhead even if e.g. update is only on R.B, not on attributes in selection condition
- Solution 2: *Defer only if required*
 - ▶ Perform immediate update if update does not affect attributes in where clause, and deferred updates otherwise.

Join Minimization

□ Join minimization

```
select r.A, r.B  
from r, s  
where r.B = s.B
```

□ Check if join with s is redundant, drop it

- E.g. join condition is on foreign key from r to s, r.B is declared as not null, and no selection on s

- Other sufficient conditions possible

```
select r.A, s2.B  
from r, s as s1, s as s2  
where r.B=s1.B and r.B = s2.B and s1.A < 20 and s2.A < 10
```

- ▶ join with s1 is redundant and can be dropped (along with selection on s1)

- Lots of research in this area since 70s/80s!

Multiquery Optimization

- Example

Q1: **select * from (r natural join t) natural join s**

Q2: **select * from (r natural join u) natural join s**

- Both queries share common subexpression (r natural join s)
- May be useful to compute (r natural join s) once and use it in both queries
 - ▶ But this may be more expensive in some situations
 - e.g. (r natural join s) may be expensive, plans as shown in queries may be cheaper
- **Multiquery optimization**: find best overall plan for a set of queries, exploiting sharing of common subexpressions between queries where it is useful

Multiquery Optimization (Cont.)

- Simple heuristic used in some database systems:
 - optimize each query separately
 - detect and exploiting common subexpressions in the individual optimal query plans
 - ▶ May not always give best plan, but is cheap to implement
 - **Shared scans**: widely used special case of multiquery optimization
- Set of materialized views may share common subexpressions
 - As a result, view maintenance plans may share subexpressions
 - Multiquery optimization can be useful in such situations

Parametric Query Optimization

- Example
select *
from r natural join s
where r.a < \$1
 - value of parameter \$1 not known at compile time
 - ▶ known only at run time
 - different plans may be optimal for different values of \$1
- Solution 1: optimize at run time, each time query is submitted
 - ▶ can be expensive
- Solution 2: **Parametric Query Optimization:**
 - optimizer generates a set of plans, optimal for different values of \$1
 - ▶ Set of optimal plans usually small for 1 to 3 parameters
 - ▶ Key issue: how to do find set of optimal plans efficiently
 - best one from this set is chosen at run time when \$1 is known
- Solution 3: **Query Plan Caching**
 - If optimizer decides that same plan is likely to be optimal for all parameter values, it caches plan and reuses it, else reoptimize each time
 - Implemented in many database systems